Cluster distances

Example from Textbook

We'll use the USArrests data as demo data sets. We'll use only a subset of the data by taking 15 random rows among the 50 rows in the data set. This is done by using the function *sample()*. Next, we standardize the data using the function *scale()*:

Scaling data

```
#Subset of the data
set.seed(123)
ss <- sample(1:50, 15) # Take 15 random rows
df <- USArrests[ss, ] # Subset the 15 rows
df.scaled <- scale(df) # Standardize the variables
```

Computing euclidean distance

We use the *dist()* function:

```
library(stats)
dist.eucl <- dist(df.scaled, method = "euclidean")
#dist.eucl
```

If needed, reformat as matrix for easier viewing

To make it easier to see the distance information generated by the *dist()* function, you can reformat the distance vector into a matrix using the *as.matrix()* function.

```
# Reformat as a matrix
# Subset the first 3 columns and rows and Round the values
round(as.matrix(dist.eucl)[1:3, 1:3], 1)
```

```
##            New Mexico Iowa Indiana
## New Mexico        0.0  4.1     2.5
## Iowa              4.1  0.0     1.8
## Indiana           2.5  1.8     0.0
```

What if you wish to cluster variables into groups?

In this data set, the columns are variables. Hence, if we want to compute pairwise distances between variables, we must start by transposing the data to have variables in the rows of the data set before using the *dist()* function. The function *t()* is used for transposing the data.

Computing correlation based distances

```
# install.packages(factoextra)
library("factoextra")
```

```
## Warning: package 'factoextra' was built under R version 4.0.5
```

```
## Loading required package: ggplot2
```

```
## Warning: package 'ggplot2' was built under R version 4.0.5
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

```
dist.cor <- get_dist(df.scaled, method = "pearson")
```

```
# Display a subset
round(as.matrix(dist.cor)[1:3, 1:3], 1)
```

```
##            New Mexico Iowa Indiana
## New Mexico        0.0  1.7     2.0
## Iowa              1.7  0.0     0.3
## Indiana           2.0  0.3     0.0
```

Computing distances for mixed data

The function *daisy()* [*cluster* package] provides a solution (Gower's metric) for computing the distance matrix, in the situation where the data contain no-numeric columns.

The R code below applies the *daisy()* function on *flower* data which contains factor, *ordered* and *numeric* variables:

```
#install.packages(cluster)
library(cluster)

# Load data
data(flower)
head(flower, 3)
```

```
##   V1 V2 V3 V4 V5 V6  V7 V8
## 1  0  1  1  4  3 15  25 15
## 2  1  0  0  2  1  3 150 50
## 3  0  1  0  3  3  1 150 50
```

```
# Data structure
str(flower)
```

```
## 'data.frame':    18 obs. of  8 variables:
##  $ V1: Factor w/ 2 levels "0","1": 1 2 1 1 1 1 1 2 2 ...
##  $ V2: Factor w/ 2 levels "0","1": 2 1 2 1 2 2 1 1 2 2 ...
##  $ V3: Factor w/ 2 levels "0","1": 2 1 1 2 1 1 1 2 1 1 ...
##  $ V4: Factor w/ 5 levels "1","2","3","4",..: 4 2 3 4 5 4 4 2 3 5 ...
##  $ V5: Ord.factor w/ 3 levels "1"<"2"<"3": 3 1 3 2 2 3 3 2 1 2 ...
##  $ V6: Ord.factor w/ 18 levels "1"<"2"<"3"<"4"<..: 15 3 1 16 2 12 13 7 4 14 ...
##  $ V7: num  25 150 150 125 20 50 40 100 25 100 ...
##  $ V8: num  15 50 50 50 15 40 20 15 15 60 ...
```

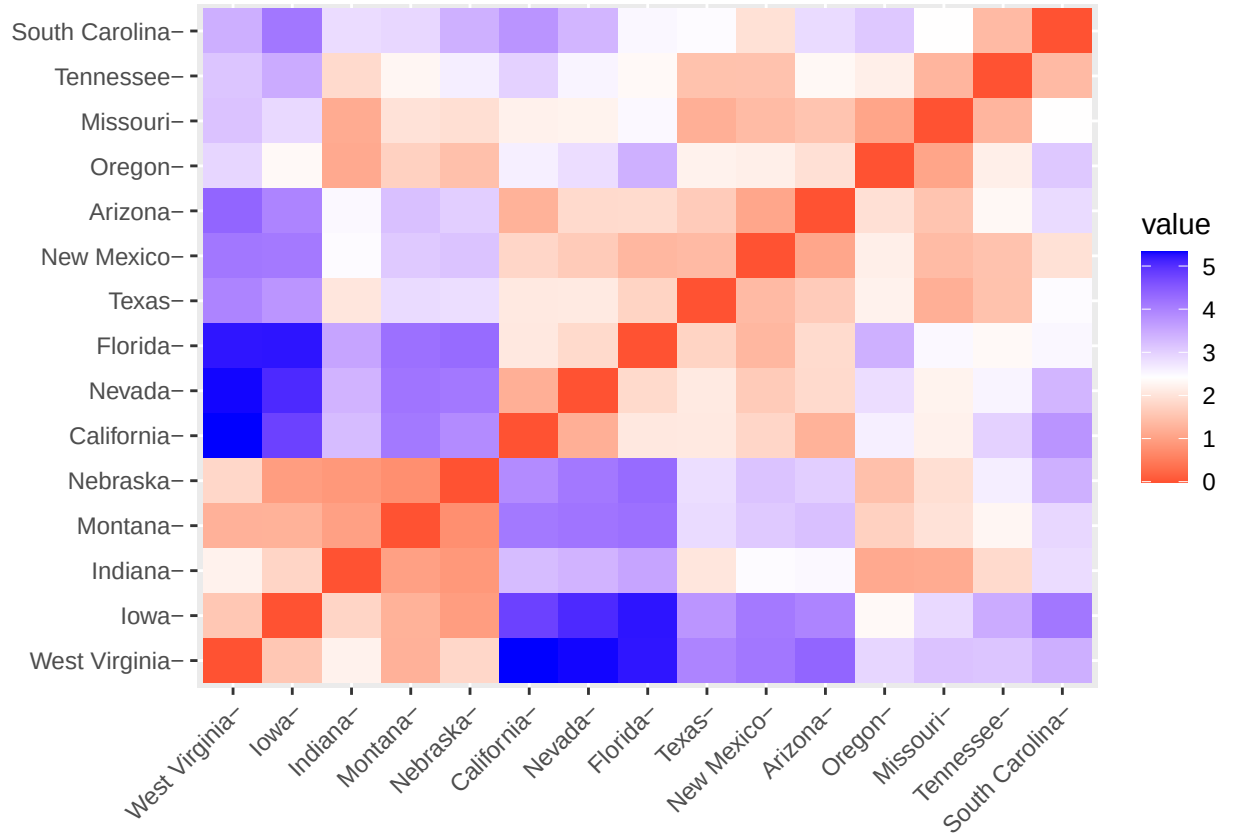
```
# Distance matrix
dd <- daisy(flower)
round(as.matrix(dd)[1:3, 1:3], 2)
```

```
##      1    2    3
## 1 0.00 0.89 0.53
## 2 0.89 0.00 0.51
## 3 0.53 0.51 0.00
```

Visualizing distance matrices

To use *fviz_dist()* type this:

```
library(factoextra)
fviz_dist(dist.eucl)
```



Red: high similarity (ie: low dissimilarity) | Blue: low similarity.